Underwater Acoustic Communications Channel Model Design

This example walks through the complete workflow for designing and simulating an underwater acoustic (UWA) communication channel for an autonomous underwater vehicle (AUV). We model the physics symbolically, build a simulation, discover the system is inadequate, and then strengthen the comms chain to meet the channel's demands.

Requires: Symbolic Math Toolbox, Communications Toolbox, Statistics and Machine Learning Toolbox

1. Symbolic Modeling of the Underwater Channel

1.1 Define Physical Parameters

The underwater acoustic channel is governed by frequency-dependent absorption, geometric spreading, and stochastic fading.

```
syms f d positive                   % frequency (kHz), distance (km)
syms k_spread positive              % spreading factor (1=cylindrical, 2=spherical)
syms S_dB real                      % source level (dB re 1 uPa)
syms N0_dBHz real                   % ambient noise spectral density (dB re 1 uPa^2/Hz)
syms BW positive                    % bandwidth (kHz)
syms gamma_s positive               % average SNR per symbol (linear)
syms m_nak positive                 % Nakagami-m fading shape parameter
syms M_order positive               % modulation order (e.g. 4 for QPSK)
```

1.2 Thorp Absorption Model

The Thorp formula gives the frequency-dependent absorption coefficient $\alpha(f)$ in dB/km for f in kHz. This is the dominant loss mechanism at ranges beyond a few hundred meters.

```
alpha_thorp = 0.11*f^2/(1 + f^2) + 44*f^2/(4100 + f^2) + 2.75e-4*f^2 + 0.003;
disp(alpha_thorp)
```

$$\frac{11\,f^2}{100\,(f^2+1)}+\frac{44\,f^2}{f^2+4100}+\frac{11\,f^2}{40000}+\frac{3}{1000}$$

The one-way transmission loss (TL) in dB combines geometric spreading and absorption:

$TL = k \cdot 10\log_{10}(d) + \alpha(f) \cdot d$

```
TL = k_spread * 10*log(d)/log(sym(10)) + alpha_thorp * d;
disp(TL)
```

$$d\left(\frac{11\,f^2}{100\,(f^2+1)}+\frac{44\,f^2}{f^2+4100}+\frac{11\,f^2}{40000}+\frac{3}{1000}\right)+\frac{10\,k_{\text{spread}}\log(d)}{\log(10)}$$

1.3 Signal-to-Noise Ratio

The noise term must use physical bandwidth in Hz. Since BW is kept in kHz for the capacity equations, the receiver noise bandwidth is $1000BW$ Hz:

$\text{SNR}_{\text{dB}} = S_{dB} - TL - N_{0,\text{dB/Hz}} - 10\log_{10}(1000BW)$

```
noiseBW_dB = 10*log(1000*BW)/log(sym(10));
SNR_dB = S_dB - TL - N0_dBHz - noiseBW_dB;
disp(SNR_dB)
```

$$S_{\text{dB}} - N_{0,\text{dBHz}} - \frac{10\log(1000\ \text{BW})}{\log(10)} - d\left(\frac{11\,f^2}{100\,(f^2+1)}+\frac{44\,f^2}{f^2+4100}+\frac{11\,f^2}{40000}+\frac{3}{1000}\right)-\frac{10\,k_{\text{spread}}\log(d)}{\log(10)}$$

Convert to linear SNR for use in error-rate expressions.

```
SNR_lin = 10^(SNR_dB/10);
```

1.4 Nakagami-m Fading: Symbol Error Rate

For a Nakagami-m fading channel with M-ary QAM the approximate symbol error rate (SER) is

$$P_s \approx \frac{2(\sqrt{M}-1)}{\sqrt{M}}\left(\frac{m}{m+\bar{\gamma}_s}\right)^m$$

The Rayleigh channel ($m = 1$) is the baseline for deep-water UWA links.

```
SER_nakagami = 2*(sqrt(M_order)-1)/sqrt(M_order) * (m_nak/(m_nak + gamma_s))^m_nak;
disp(SER_nakagami)
```

$$\frac{\left(\frac{m_{\text{nak}}}{\gamma_s + m_{\text{nak}}}\right)^{m_{\text{nak}}} \left(2\sqrt{M_{\text{order}}} - 2\right)}{\sqrt{M_{\text{order}}}}$$

Substitute the channel conditions: $m = 1$ (Rayleigh), $M = 4$ (QPSK).

```
SER_rayleigh = subs(SER_nakagami, [m_nak, M_order], [1, 4]);
SER_rayleigh = simplify(SER_rayleigh);
disp(SER_rayleigh)
```

$$\frac{1}{\gamma_s + 1}$$

1.5 Channel Capacity: AWGN Bound and Rayleigh Capacity

The AWGN expression is an upper bound on the fading link:

$C_{\text{AWGN}} = BW \cdot \log_2(1 + \bar{\gamma}_s)$

```
C_awgn = BW * log(1 + gamma_s)/log(sym(2));
disp(C_awgn)
```

$$\frac{\text{BW}\,\log(\gamma_s + 1)}{\log(2)}$$

Isolate SNR for a target capacity C_t under the AWGN upper bound.

```
syms C_t positive
SNR_for_capacity_awgn = isolate(C_awgn == C_t, gamma_s);
disp(SNR_for_capacity_awgn)
```

$$\gamma_s = 2^{C_t/\text{BW}} - 1$$

Solve for required bandwidth given target capacity and SNR.

```
BW_required_awgn = isolate(C_awgn == C_t, BW);
```

```
disp(BW_required_awgn)
```

$$\mathrm{BW} = \frac{C_t \log(2)}{\log(\gamma_s + 1)}$$

For coherent Rayleigh fading, the instantaneous fade power z is exponentially distributed. The fading capacity is the expected value of the AWGN capacity over that distribution:

$$C_{\mathrm{Rayleigh}} = \frac{BW}{\ln 2} \int_0^{\infty} \ln(1 + \bar{\gamma}_s z) e^{-z}\, dz$$

```
syms z positive
fadePower_pdf = exp(-z);
C_rayleigh_integral = BW/log(sym(2)) * int(log(1 + gamma_s*z)*fadePower_pdf, z, 0, Inf);
disp(C_rayleigh_integral)
```

$$-\frac{\mathrm{BW}\ \mathrm{e}^{1/\gamma_s}\ \mathrm{ei}\left(-\frac{1}{\gamma_s}\right)}{\log(2)}$$

For numeric handoff, use the equivalent positive-SNR closed form with MATLAB's `expint`:

```
C_rayleigh = BW * exp(1/gamma_s) * expint(1/gamma_s) / log(sym(2));
disp(C_rayleigh)
```

$$\frac{\mathrm{BW}\ \mathrm{e}^{1/\gamma_s}\ \mathrm{expint}\left(\frac{1}{\gamma_s}\right)}{\log(2)}$$

1.6 Sensitivity Analysis via Gradient

Quantify how sensitive the SER is to small changes in SNR. This is a key advantage of the symbolic approach - the closed-form gradient tells us exactly how fragile the link is, whereas a numeric simulation can only approximate by sweeping parameters.

```
grad_SER = gradient(SER_rayleigh, gamma_s);
disp(grad_SER)
```

$$-\frac{1}{(\gamma_s+1)^2}$$

Normalised sensitivity (elasticity): $E = \frac{\partial P_s}{\partial \bar{\gamma}_s} \cdot \frac{\bar{\gamma}_s}{P_s}$

```
elasticity_SER = simplify(grad_SER * gamma_s / SER_rayleigh);
disp(elasticity_SER)
```

$$-\frac{\gamma_s}{\gamma_s+1}$$

The elasticity tells us the *percentage* change in SER per 1 % change in SNR. At low SNR this will be close to -1 (Rayleigh), confirming the channel is harsh and the system needs strong coding.

1.7 Maximum Code Rate from Capacity Limits

```
syms R_code positive
```

A code can succeed only if its spectral efficiency stays below the channel capacity. First solve the optimistic AWGN limit:

```
maxRate_awgn_sym = isolate(R_code * log(M_order)/log(sym(2)) == log(1 + gamma_s)/log(sym(2)), R_code);
disp(maxRate_awgn_sym)
```

$$R_{\text{code}} = \frac{\log(\gamma_s+1)}{\log(M_{\text{order}})}$$

Then compute the Rayleigh-fading limit, which is the bound used for the design decision:

```
maxRate_rayleigh_sym = simplify(C_rayleigh / (BW * log(M_order)/log(sym(2))));
disp(maxRate_rayleigh_sym)
```

$$\frac{e^{1/\gamma_s}\,\text{expint}\left(\frac{1}{\gamma_s}\right)}{\log(M_{\text{order}})}$$

By substituting our actual SNR, we can determine *before any simulation* whether a given code rate is viable for the fading channel.

1.8 3-D Performance Surface

Use `fmesh` to plot SER as a function of distance and frequency, substituting concrete values for all other parameters. This surface reveals the "sweet spot" where performance is best, and exposes the failure region our system must avoid.

Fix source level at 150 dB (small AUV transducer), ambient noise density at 44 dB re 1 uPa^2/Hz, bandwidth 10 kHz, spreading factor 1.5, $m = 1$ (Rayleigh).

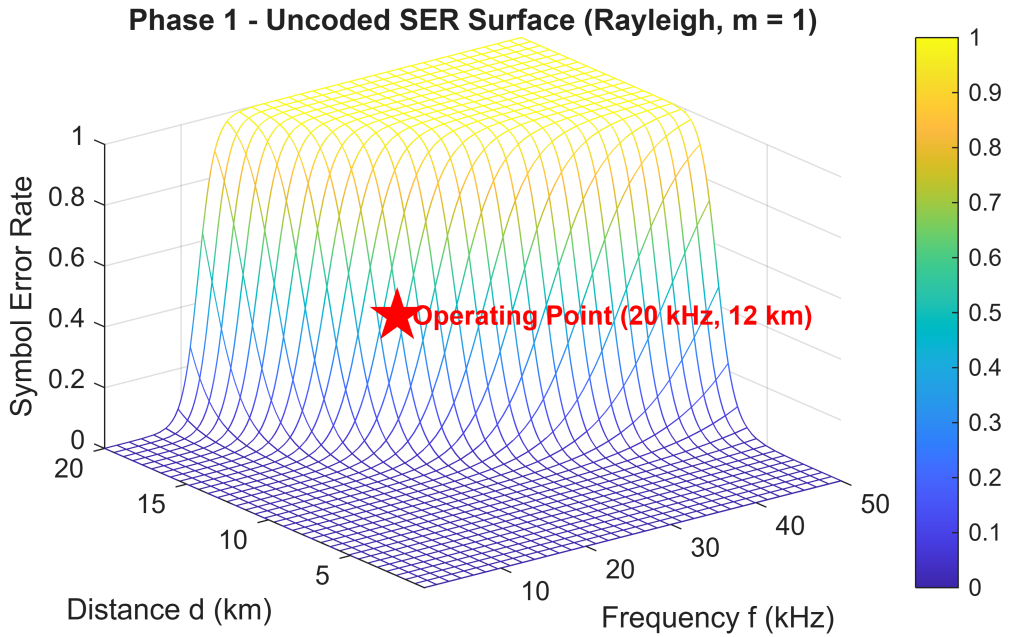
```
SER_surface_expr = subs(SER_nakagami, m_nak, 1);
SER_surface_expr = subs(SER_surface_expr, M_order, 4);
```

Build the full SNR chain: substitute the Thorp + TL model into SNR, then into SER.

```
SNRVal = subs(SNR_dB, [S_dB, N0_dBHz, BW, k_spread], [150, 44, 10, 1.5]);
gammaVal = 10^(SNRVal/10);
SER_full = subs(SER_surface_expr, gamma_s, gammaVal);
SER_full = simplify(SER_full);
```

Plot the surface and mark the operating point (20 kHz, 12 km) where our AUV will operate. The surface already predicts this point sits in a high-error region. Phase 2 will confirm this.

```
fmesh(SER_full, [1 50 0.5 20])
hold on
SER_at_op = double(subs(SER_full, [f, d], [20, 12]));
plot3(20, 12, SER_at_op, 'rp', 'MarkerSize', 20, 'MarkerFaceColor', 'r')
text(20, 12, SER_at_op, '  Operating Point (20 kHz, 12 km)', ...
    'FontSize', 10, 'Color', 'r', 'FontWeight', 'bold')
hold off
xlabel('Frequency f (kHz)')
ylabel('Distance d (km)')
zlabel('Symbol Error Rate')
title('Phase 1 - Uncoded SER Surface (Rayleigh, m = 1)')
colorbar
```



1.9 Convert Symbolic Formulas to Numeric Functions

Convert key expressions into MATLAB function handles for use in the simulation phases. `matlabFunction` bridges the symbolic and numeric worlds, so every formula derived above becomes callable in the Communications Toolbox chain.

```
SER_nakagami_fn = matlabFunction(SER_nakagami, 'Vars', [gamma_s, m_nak, M_order]);
alpha_fn = matlabFunction(alpha_thorp, 'Vars', f);
TL_fn = matlabFunction(TL, 'Vars', [f, d, k_spread]);
maxRate_rayleigh_fn = matlabFunction(maxRate_rayleigh_sym, 'Vars', [gamma_s, M_order]);
```

2. Communications Simulation (Rate 3/4 LDPC)

Build the actual transmit/receive chain for our AUV-to-buoy link using Communications Toolbox. For this first attempt we deliberately choose an optimistically weak code (3/4 LDPC with no inner code) to see what happens when the coding does not match the channel severity.

2.1 Simulation Parameters

Model an autonomous underwater vehicle (AUV) communicating with a surface buoy. The small transducer limits source level, and the 12 km patrol range through elevated ambient noise puts the link near 0 dB SNR, where weak codes fail but stronger low-rate codes remain plausible.

```
numBits    = 5e5;
modOrder   = 4;             % QPSK
fc_kHz     = 20;            % carrier frequency in kHz (mid-frequency UWA modem)
dist_km    = 12;            % link distance in km (FIXED for both passes)
kSpread    = 1.5;           % spreading factor (practical)
srcLevel   = 150;           % source level dB re 1 uPa (~small AUV transducer)
noiseDensity = 44;          % ambient noise spectral density dB re 1 uPa^2/Hz
bw_kHz     = 10;            % bandwidth kHz
nakagami_m_val = 1;         % Rayleigh fading (fixed channel assumption)
```

2.2 Compute Numeric SNR from Symbolic Model

The function handles from Phase 1 compute the exact SNR, so no need to re-derive the physics numerically.

```
snr_dB_Val = srcLevel ...
    - double(TL_fn(fc_kHz, dist_km, kSpread)) ...
    - noiseDensity ...
    - 10*log10(1000*bw_kHz);
snr_lin_Val = 10^(snr_dB_Val/10);
integratedNoise_dB = noiseDensity + 10*log10(1000*bw_kHz);
disp(['Integrated noise level: ', num2str(integratedNoise_dB, '%.1f'), ' dB re 1 uPa'])
```

```
Integrated noise level: 84.0 dB re 1 uPa
```

```
disp(['Received SNR: ', num2str(snr_dB_Val, '%.1f'), ' dB'])
```

```
Received SNR: 0.2 dB
```

Check the capacity limits. Substitute our numeric SNR into the symbolic max-rate formulas:

```
maxRate_awgn_numeric = log2(1 + snr_lin_Val) / log2(modOrder);
maxRate_numeric = maxRate_rayleigh_fn(snr_lin_Val, modOrder);
fprintf(['AWGN upper-bound max code rate (QPSK): %.3f\n' ...
    'Rayleigh fading max code rate (QPSK) : %.3f\n' ...
    '  >> If our code rate exceeds this, reliable comms are impossible.\n'], ...
```

```
    maxRate_awgn_numeric, maxRate_numeric)
```

```
AWGN upper-bound max code rate (QPSK): 0.517
Rayleigh fading max code rate (QPSK) : 0.444
  >> If our code rate exceeds this, reliable comms are impossible.
```

2.3 CRC Generator for Error Detection

A cyclic redundancy check appends parity bits so the receiver can *detect* residual errors after decoding.

```
crcGen = crcConfig(Polynomial='z^16+z^15+z^2+1');
crcLen = 16;
```

2.4 LDPC Encoder / Decoder (Rate 3/4)

Use a standard DVB-S.2 LDPC code. Start with **rate 3/4**, which is an aggressive choice. The symbolic analysis in Phase 1 already warned that this will be tight.

```
ldpcEncCfg = ldpcEncoderConfig(dvbsLDPCPCM('3/4'));
ldpcDecCfg = ldpcDecoderConfig(ldpcEncCfg, 'norm-min-sum');
maxNumIter = 50;
infoLen   = ldpcEncCfg.NumInformationBits;
codeLen   = ldpcEncCfg.BlockLength;
codeRate  = infoLen / codeLen;
disp(['LDPC code rate: ', num2str(codeRate)])
```

```
LDPC code rate: 0.75
```

2.5 Transmit Chain (Rate 3/4 LDPC)

Generate random bits, apply CRC, encode with LDPC, modulate QPSK, and send the symbols through the same Rayleigh + AWGN receiver used for both code rates.

```
[BER_pass1, numErrors_pass1, numBits_pass1, txBitsAll, rxBitsAll] = simulateLdpcRayleigh( ...
    numBits, ldpcEncCfg, ldpcDecCfg, crcGen, crcLen, modOrder, ...
    snr_dB_Val, nakagami_m_val, maxNumIter, 42);
```

2.6 Bit-Error Rate (Rate 3/4 LDPC)

```
fprintf(['=== RATE 3/4 LDPC (Rayleigh, %.0f km) ===\n' ...
    '    BER: %.4e\n' ...
    '    Errors: %d / %d\n'], ...
    dist_km, BER_pass1, numErrors_pass1, numBits_pass1)
```

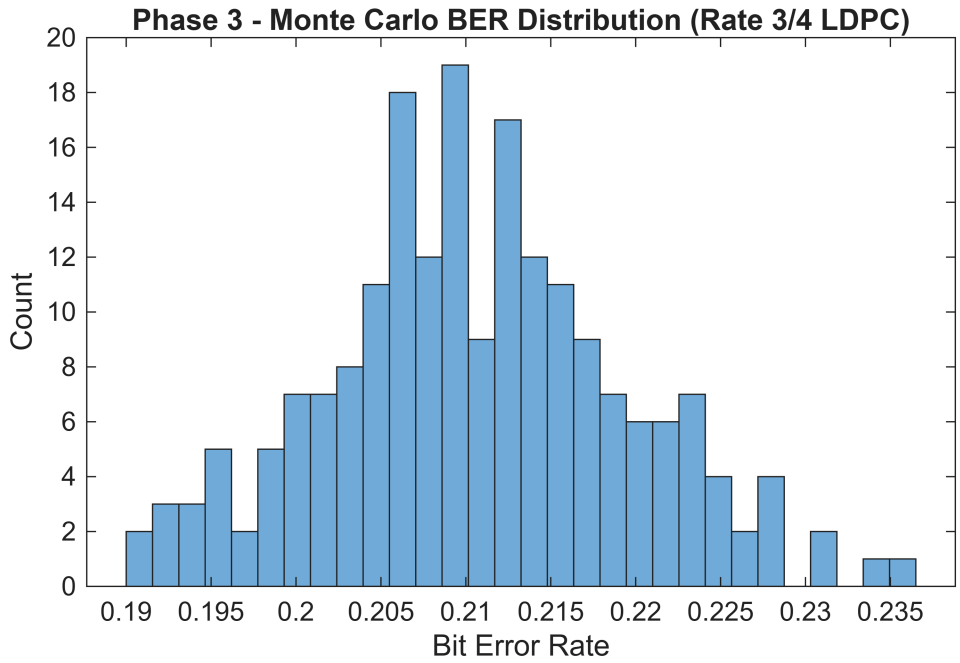
```
=== RATE 3/4 LDPC (Rayleigh, 12 km) ===
    BER: 2.1243e-01
    Errors: 103240 / 486000
```

3. Diagnosis

3.1 Monte Carlo BER Variance

Draw random batches of received bits and estimate BER variance

```
numTrials  = 200;
batchSize  = 2000;
berSamples = zeros(numTrials, 1);
for t = 1:numTrials
    idx = datasample(1:numel(txBitsAll), batchSize);
    berSamples(t) = mean(txBitsAll(idx) ~= rxBitsAll(idx));
end
histogram(berSamples, 30)
xlabel('Bit Error Rate')
ylabel('Count')
title('Phase 3 - Monte Carlo BER Distribution (Rate 3/4 LDPC)')
```




```
fprintf(['Monte Carlo mean BER : %.4e\n' ...
    'Monte Carlo std BER  : %.4e\n'], ...
    mean(berSamples), std(berSamples))
```

```
Monte Carlo mean BER : 2.1064e-01
Monte Carlo std BER  : 8.8471e-03
```

3.2 Confirming the Symbolic Analysis

The high BER confirms what the Phase 1 surface plot already showed: our operating point (20 kHz, 12 km) sits in the high-error region. The rate-3/4 code cannot compensate for Rayleigh fading at this SNR. The fix is to strengthen the comms chain. Return to the symbolic model to determine exactly how much stronger.

```
fprintf(['=== RATE 3/4 LDPC DIAGNOSIS ===\n' ...
    '  Post-LDPC BER         : %.4e\n' ...
    '  Rayleigh max code rate: %.3f\n' ...
    '  Actual code rate      : %.3f\n\n' ...
```

```
    '  >> VERDICT: Rate 3/4 LDPC is too weak for this channel.\n' ...
    '  >> ACTION: Return to symbolic model, compute required code rate,\n' ...
    '     then rebuild the comms chain with stronger FEC.\n'], ...
    BER_pass1, maxRate_numeric, codeRate)
```

```
=== RATE 3/4 LDPC DIAGNOSIS ===
  Post-LDPC BER         : 2.1243e-01
  Rayleigh max code rate: 0.444
  Actual code rate      : 0.750

  >> VERDICT: Rate 3/4 LDPC is too weak for this channel.
  >> ACTION: Return to symbolic model, compute required code rate,
     then rebuild the comms chain with stronger FEC.
```

4. Strengthen the Comms Chain

4.1 Return to Symbolic Analysis

Rayleigh fading at 12 km is the physical reality. Return to our symbolic equations to determine what code rate the system actually needs.

From Section 1.7, the symbolic Rayleigh-fading maximum code rate is:

```
disp(maxRate_rayleigh_sym)
```

$$\frac{e^{1/\gamma_s}\,\mathrm{expint}\left(\frac{1}{\gamma_s}\right)}{\log(M_{\mathrm{order}})}$$

Substitute our actual SNR to get the numeric bound:

```
maxRate_at_SNR = maxRate_rayleigh_fn(snr_lin_Val, modOrder);
fprintf(['Channel SNR: %.1f dB (linear: %.2f)\n' ...
    'Maximum Rayleigh code rate (symbolic -> numeric): %.3f\n' ...
    'First-pass code rate was               : %.3f\n' ...
    '  >> The symbolic model confirms: rate 3/4 exceeds the capacity limit.\n' ...
    '  >> Strategy: drop to a DVB-S.2 rate 1/4 LDPC code.\n'], ...
    snr_dB_Val, snr_lin_Val, maxRate_at_SNR, codeRate)
```

```
Channel SNR: 0.2 dB (linear: 1.05)
Maximum Rayleigh code rate (symbolic -> numeric): 0.444
```

```
First-pass code rate was               : 0.750
  >> The symbolic model confirms: rate 3/4 exceeds the capacity limit.
  >> Strategy: drop to a DVB-S.2 rate 1/4 LDPC code.
```

4.2 Re-run Phase 2 with Stronger Coding

What changed:

- LDPC code rate lowered from 3/4 to **1/4** (more redundancy)
- Soft LLR demodulation and LDPC decoding are retained end to end
- Same Rayleigh channel, same distance, same SNR

```
ldpcEncCfg_v2 = ldpcEncoderConfig(dvbsLDPCPCM('1/4'));
ldpcDecCfg_v2 = ldpcDecoderConfig(ldpcEncCfg_v2, 'norm-min-sum');
infoLen_v2  = ldpcEncCfg_v2.NumInformationBits;
codeLen_v2  = ldpcEncCfg_v2.BlockLength;
codeRate_v2 = infoLen_v2 / codeLen_v2;
fprintf(['LDPC code rate: %.3f\n' ...
    'Channel: SAME (Rayleigh m=1, d=%.0f km, SNR=%.1f dB)\n'], ...
    codeRate_v2, dist_km, snr_dB_Val)
```

```
LDPC code rate: 0.250
Channel: SAME (Rayleigh m=1, d=12 km, SNR=0.2 dB)
```

4.3 Transmit / Receive (Rate 1/4 LDPC)

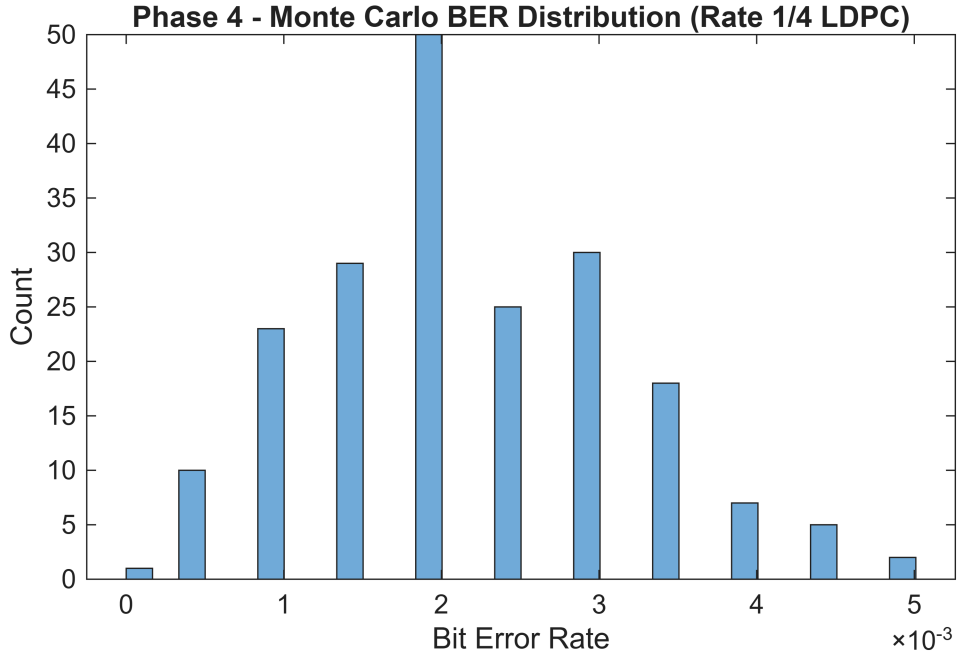
Reuse the same channel helper and change only the code rate.

```
[BER_pass2, numErrors_pass2, numBits_pass2, txBitsAll_v2, rxBitsAll_v2] = simulateLdpcRayleigh( ...
    numBits, ldpcEncCfg_v2, ldpcDecCfg_v2, crcGen, crcLen, modOrder, ...
    snr_dB_Val, nakagami_m_val, maxNumIter, 42);
fprintf(['=== RATE 1/4 LDPC (Rayleigh, %.0f km) ===\n' ...
    '    BER: %.4e\n' ...
    '    Errors: %d / %d\n'], ...
    dist_km, BER_pass2, numErrors_pass2, numBits_pass2)
```

```
=== RATE 1/4 LDPC (Rayleigh, 12 km) ===
    BER: 2.3724e-03
    Errors: 1153 / 486000
```

4.4 Monte Carlo (Rate 1/4 LDPC)

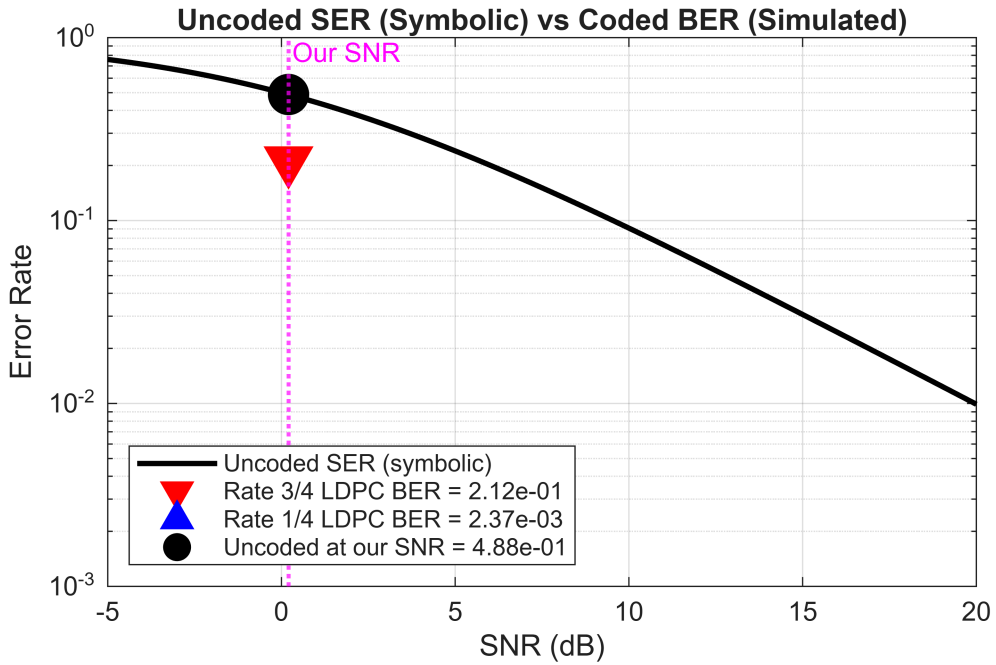
```
berSamples_v2 = zeros(numTrials, 1);
for t = 1:numTrials
    idx = datasample(1:numel(txBitsAll_v2), min(batchSize, numel(txBitsAll_v2)));
    berSamples_v2(t) = mean(txBitsAll_v2(idx) ~= rxBitsAll_v2(idx));
end
histogram(berSamples_v2, 30)
xlabel('Bit Error Rate')
ylabel('Count')
title('Phase 4 - Monte Carlo BER Distribution (Rate 1/4 LDPC)')
```



4.5 Uncoded SER (Symbolic) with Simulated Coded BER

The symbolic Rayleigh SER curve is the analytical baseline - it applies to uncoded transmission. The coded BER points come from the Communications Toolbox simulation. The gap between the uncoded curve and the simulation markers shows the coding gain achieved. The symbolic analysis told us which code rate to use, while the simulation confirms how well it works.

```
snr_range = linspace(-5, 20, 200);
ser_uncoded = SER_nakagami_fn(10.^(snr_range/10), 1, 4);
semilogy(snr_range, ser_uncoded, 'k-', 'LineWidth', 2)
hold on
plot(snr_dB_Val, BER_pass1, 'rv', 'MarkerSize', 14, 'MarkerFaceColor', 'r', 'LineWidth', 1.5)
plot(snr_dB_Val, max(BER_pass2, 1e-6), 'b^', 'MarkerSize', 14, 'MarkerFaceColor', 'b', 'LineWidth', 1.5)
SER_uncoded_at_op = SER_nakagami_fn(snr_lin_Val, 1, 4);
plot(snr_dB_Val, SER_uncoded_at_op, 'ko', 'MarkerSize', 14, 'MarkerFaceColor', 'k', 'LineWidth', 1.5)
xline(snr_dB_Val, 'm:', 'Our SNR', 'LineWidth', 1.5, 'LabelOrientation', 'horizontal')
hold off
xlabel('SNR (dB)')
ylabel('Error Rate')
title('Uncoded SER (Symbolic) vs Coded BER (Simulated)')
legend('Uncoded SER (symbolic)', ...
    ['Rate 3/4 LDPC BER = ', num2str(BER_pass1, '%.2e')], ...
    ['Rate 1/4 LDPC BER = ', num2str(BER_pass2, '%.2e')], ...
    ['Uncoded at our SNR = ', num2str(SER_uncoded_at_op, '%.2e')], ...
    'Location', 'southwest')
grid on
```



4.6 Final Comparison

Side-by-side summary: same channel, improved comms chain. The symbolic capacity analysis predicted the failure (rate 3/4 exceeds the Rayleigh capacity limit), diagnosed the required code rate, and the simulation confirmed the fix.

```
fprintf(['============================================================\n' ...
    '     UNDERWATER ACOUSTIC LINK - FINAL RESULTS\n' ...
    '     (Same channel: Rayleigh m=1, %.0f km, %.0f kHz)\n' ...
    '============================================================\n' ...
    '  Rayleigh max code rate (symbolic): %.3f\n' ...
    '  Rate 3/4 LDPC          : %.3f  >> EXCEEDS limit\n' ...
    '  Rate 1/4 LDPC          : %.3f  >> Below limit\n' ...
    '------------------------------------------------------------\n' ...
    '  Rate 3/4 LDPC       BER  = %.4e\n' ...
    '  Rate 1/4 LDPC       BER  = %.4e\n' ...
    '------------------------------------------------------------\n' ...
    '  Rate 3/4 Monte Carlo BER = %.4e\n' ...
```

```
    '  Rate 1/4 Monte Carlo BER = %.4e\n' ...
    '============================================================\n'], ...
    dist_km, fc_kHz, maxRate_at_SNR, codeRate, codeRate_v2, ...
    BER_pass1, BER_pass2, mean(berSamples), mean(berSamples_v2))
```

```
============================================================
    UNDERWATER ACOUSTIC LINK - FINAL RESULTS
    (Same channel: Rayleigh m=1, 12 km, 20 kHz)
============================================================
  Rayleigh max code rate (symbolic): 0.444
  Rate 3/4 LDPC          : 0.750  >> EXCEEDS limit
  Rate 1/4 LDPC          : 0.250  >> Below limit
------------------------------------------------------------
  Rate 3/4 LDPC       BER  = 2.1243e-01
  Rate 1/4 LDPC       BER  = 2.3724e-03
------------------------------------------------------------
  Rate 3/4 Monte Carlo BER = 2.1064e-01
  Rate 1/4 Monte Carlo BER = 2.2375e-03
============================================================
```

Simulation Helper

This helper keeps the main workflow focused on the symbolic derivation and handoff. It uses unit-power QPSK, complex Rayleigh fading, explicit AWGN variance, zero-forcing equalization, and soft LDPC decoding.

```
function [BER, numErrors, totalBits, txBitsAll, rxBitsAll] = simulateLdpcRayleigh(numBits, ldpcEncCfg,
ldpcDecCfg, crcGen, crcLen, modOrder, snr_dB_Val, nakagami_m_val, maxNumIter, seed)
rng(seed);
infoLenLocal = ldpcEncCfg.NumInformationBits;
numCodewordsLocal = floor(numBits / infoLenLocal);
txBitsAll = false(numCodewordsLocal*infoLenLocal, 1);
rxBitsAll = false(numCodewordsLocal*infoLenLocal, 1);
for cw = 1:numCodewordsLocal
    dataBits = randi([0 1], infoLenLocal - crcLen, 1);
    crcBits = crcGenerate(dataBits, crcGen);
    codedBits = ldpcEncode(crcBits, ldpcEncCfg);
    txSym = pskmod(codedBits, modOrder, InputType='bit');
    fadePower = gamrnd(nakagami_m_val, 1/nakagami_m_val, size(txSym));
    h = sqrt(fadePower) .* exp(1i*2*pi*rand(size(txSym)));
    [rxSym, noisePower] = awgn(h .* txSym, snr_dB_Val, 0);
    eqSym = rxSym ./ h;
```

```
    eqNoisePower = noisePower ./ max(abs(h).^2, eps);
    softBitMatrix = pskdemod(eqSym.', modOrder, OutputType='llr', ...
        NoiseVariance=eqNoisePower.');
    softBits = softBitMatrix(:);
    decBits = ldpcDecode(softBits, ldpcDecCfg, maxNumIter);
    bitIdx = (cw-1)*infoLenLocal + (1:infoLenLocal);
    txBitsAll(bitIdx) = logical(crcBits);
    rxBitsAll(bitIdx) = logical(decBits);
end
numErrors = sum(txBitsAll ~= rxBitsAll);
totalBits = numel(txBitsAll);
BER = numErrors / totalBits;
end
```